Vulnerability Scanning and Security Automation using Tenable Nessus

NIS IA Project Report

Course Name: Network & Information Security

Group Members:
Anthony Jennis Nadar - 16010123051
Anuj Madke - 16010123052
Anurag Singh - 16010123053

Date: March 28, 2026

A comprehensive analysis of unauthenticated vs. authenticated vulnerability scanning, advanced plugin configuration, and REST API automation.

Contents

1 Introduction **2**

2 Target Environment & Isolation Setup **3**

3 Scanner Installation and Configuration **4**

4 Reconnaissance: Host Discovery Scanning **5**

5 Comparative Analysis: Scanning Methodologies **6**

5.1 Unauthenticated Network Scan (Black-Box) 6

5.2 Authenticated Network Scan (White-Box) 6

6 Advanced Scan Configuration & Optimization **8**

7 Vulnerability Analysis & Remediation Strategies **9**

7.1 Finding 1: VNC Server 'password' Password (CVSS 10.0) 9

7.2 Finding 2: Bind Shell Backdoor (CVSS 9.8) 9

8 DevSecOps: Security Automation via Nessus REST API **11**

9 Conclusion **13**

10 References **14**

1 Introduction

In the modern cybersecurity landscape, proactive Vulnerability Management is a critical pillar of Network and Information Security (NIS). Identifying misconfigurations, unpatched software, and weak access controls before malicious actors exploit them is essential for maintaining the confidentiality, integrity, and availability (CIA) of enterprise systems.

For this project, our team selected **Tenable Nessus Essentials**, an industry-leading Dynamic Application Security Testing (DAST) and infrastructure scanning tool. The objective of this report is to demonstrate the end-to-end lifecycle of a vulnerability assessment. This includes establishing an isolated target environment, conducting network reconnaissance, comparing the efficacy of credentialed versus non-credentialed scans, performing detailed vulnerability analysis, and utilizing Python to automate security workflows via the Nessus REST API.

2 Target Environment & Isolation Setup

Scanning public networks or university infrastructure without explicit written authorization violates strict ethical hacking guidelines and computer misuse laws. Therefore, a safe, isolated, and legally compliant testing environment was engineered using virtualization.

We deployed **Metasploitable 2**, an Ubuntu Linux-based virtual machine intentionally designed by Rapid7 to be highly vulnerable. The hypervisor used was Oracle VM VirtualBox.

To ensure the vulnerabilities did not leak onto the public internet and to protect our host machine, the network interface of the Virtual Machine (VM) was strictly configured to a **Host-Only Adapter**. This allowed our host Windows machine to communicate with the target over a private subnet (192.168.56.x) while dropping all external routing.

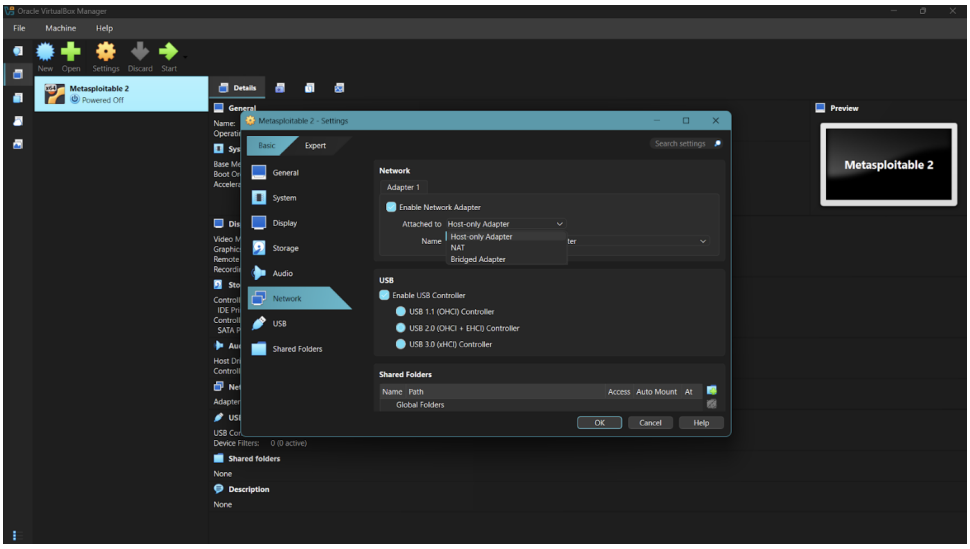

Figure 1: VirtualBox network configuration showing the strict Host-Only Adapter isolation.

Following the successful boot of the Metasploitable VM, we verified network connectivity. By authenticating into the Linux terminal (using default credentials `msfadmin:msfadmin`) and executing the `ifconfig` command, we identified the target's IP address as **192.168.56.101**. A subsequent `ping` command from the Windows host command prompt resulted in a successful 0% packet loss, confirming the testing lab was operational.

3 Scanner Installation and Configuration

With the target prepared, we registered for an educational license of Tenable Nessus Essentials. The software was installed on the Windows host machine, which included the necessary packet capture drivers (WinPcap/Npcap) required for deep network probing.

Upon completion, the Nessus web daemon was launched, accessible locally via port 8834 (`https://localhost:8834`). After applying the 20-digit activation code, Nessus initiated a lengthy process of downloading and compiling its latest vulnerability plugin database. An administrative account was created to manage scan policies and export reports.

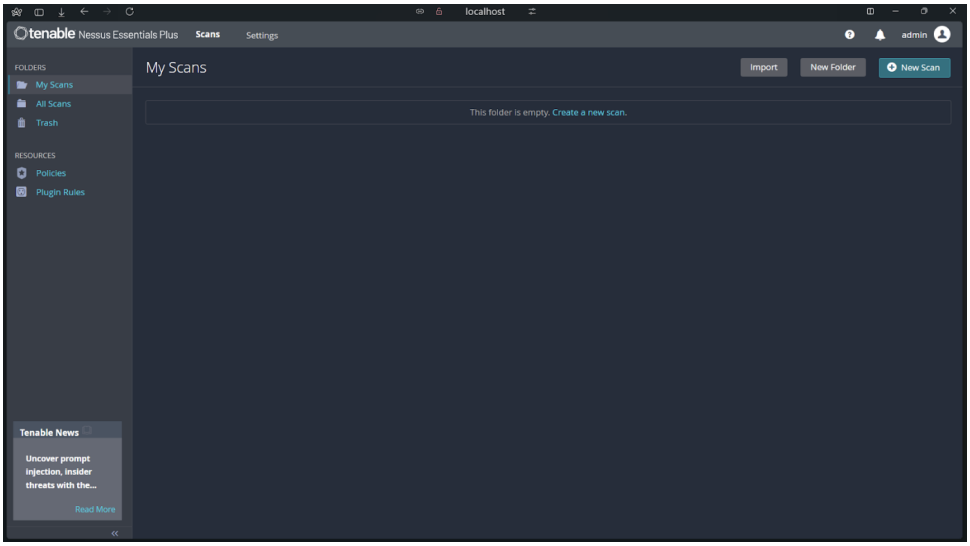

Figure 2: The initialized Nessus Essentials dashboard, ready for scan configuration.

4 Reconnaissance: Host Discovery Scanning

The first technical phase of the assessment was reconnaissance. Before querying for vulnerabilities, a security tool must map the topology of the network to see which IP addresses are active and what basic services are responding.

We created a new scan utilizing the **Host Discovery** template. Targeting `192.168.56.101`, this scan utilized ICMP ping requests and basic TCP SYN sweeps to verify the host's status. The scan completed in approximately 2 minutes, successfully detecting the host and enumerating a vast array of open ports including 111 (RPC), 139 (NetBIOS), and 445 (SMB).

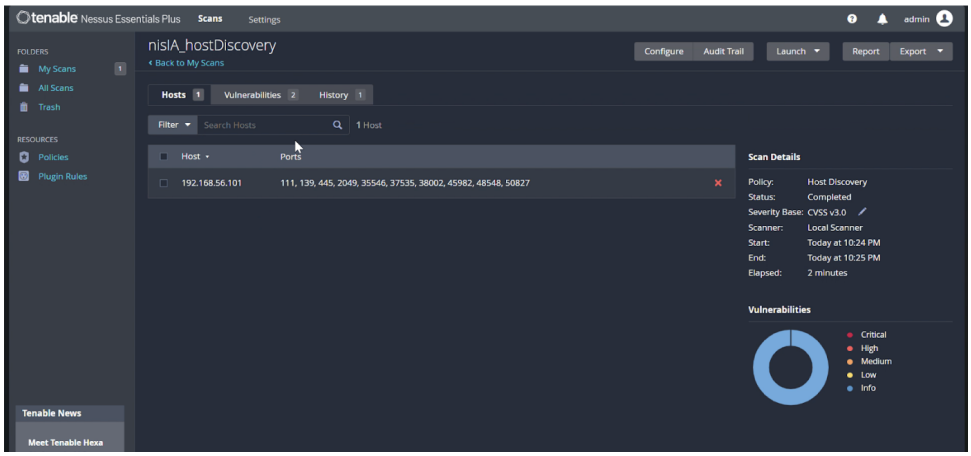

Figure 3: Host Discovery results confirming the live target and initially detected open ports.

5 Comparative Analysis: Scanning Methodologies

A primary objective of this project was to visually and technically compare the two main methodologies of vulnerability scanning: Unauthenticated (Black-Box) and Authenticated (White-Box).

5.1 Unauthenticated Network Scan (Black-Box)

An unauthenticated scan simulates an external threat actor who has no prior access or credentials to the system. The scanner is forced to rely entirely on analyzing network traffic, banner grabbing (reading the headers of web servers or FTP servers), and probing open ports to guess what software might be running.

We executed a "Basic Network Scan" without providing any login details. As expected, the results were somewhat limited. While it did find vulnerabilities based on outdated service banners, it completely lacked visibility into the internal file system, resulting in a low count of Critical vulnerabilities.

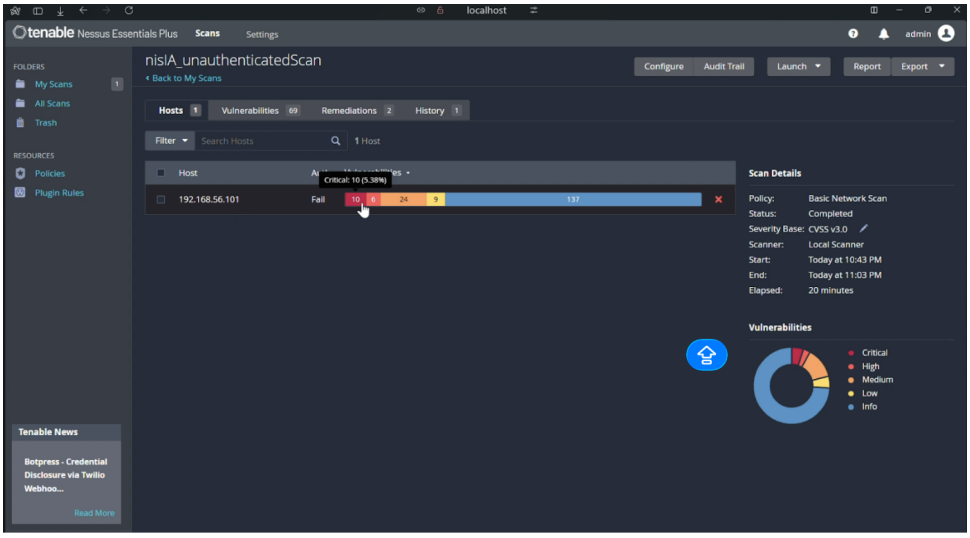

Figure 4: Unauthenticated scan results: Nessus acts as an outsider, finding limited vulnerabilities (10 Critical).

5.2 Authenticated Network Scan (White-Box)

To demonstrate a comprehensive internal audit, we duplicated the previous scan but navigated to the "Credentials" tab. Here, we provided Nessus with the SSH login credentials for the target Linux machine (`msfadmin:msfadmin`).

This fundamentally changed how Nessus operated. Instead of guessing from the outside, Nessus securely logged into the Linux kernel via SSH. It executed local commands to check the exact versions of installed packages, analyzed local configuration files (like `/etc/shadow`), and reviewed registry equivalents. The result was a massive increase in

threat visibility, uncovering **27 Critical vulnerabilities** that were completely hidden during the unauthenticated phase.

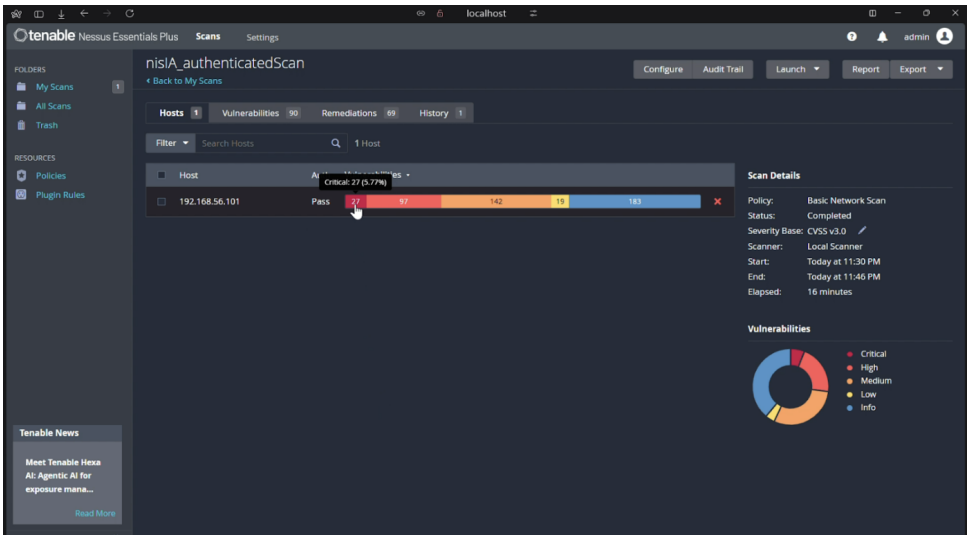

Figure 5: Authenticated scan results: Utilizing SSH credentials provides deep system visibility, revealing 27 Critical threats.

6 Advanced Scan Configuration & Optimization

Enterprise environments cannot always afford to run full, aggressive scans across all ports and plugins, as it generates massive network noise and can disrupt fragile legacy systems. To demonstrate scan optimization, we utilized the **Advanced Scan** template.

By navigating to the "Plugins" menu, we were presented with Nessus's entire database of threat checks. We disabled all plugins globally, and strategically re-enabled only a select few families relevant to our specific goal (e.g., Web Servers and CGI abuses). This targeted approach reduced the scan execution time significantly (completed in 11 minutes) and minimized the load on the target server.

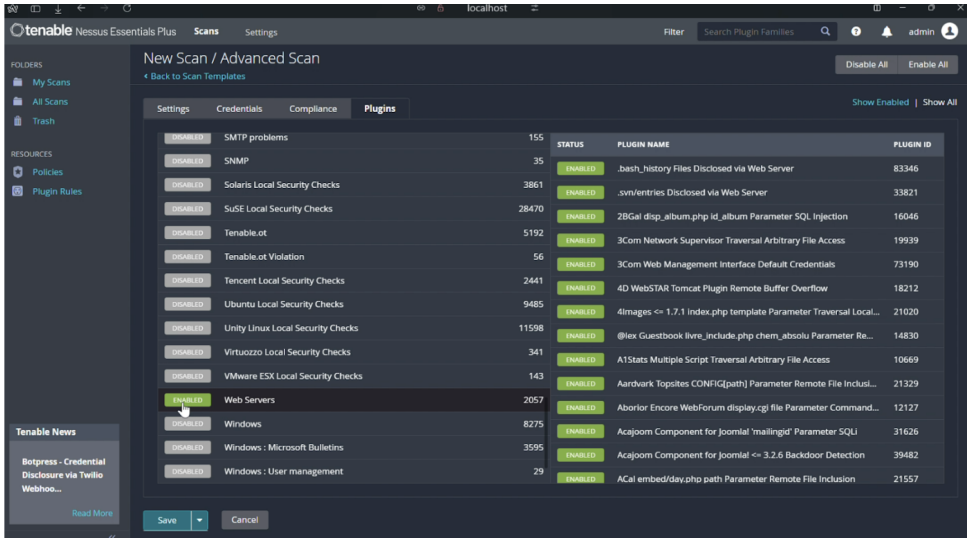

Figure 6: Customizing plugin families in the Advanced Scan to target specific threat vectors and optimize performance.

7 Vulnerability Analysis & Remediation Strategies

Identifying a vulnerability is only the first step; analyzing the risk and applying remediation is the core of NIS. Below is an analysis of two critical vulnerabilities extracted from our Authenticated Scan report.

7.1 Finding 1: VNC Server 'password' Password (CVSS 10.0)

Description: Nessus flagged that the Virtual Network Computing (VNC) server on port 5900 is secured with a trivial default password (the word "password"). VNC is a remote desktop protocol.
Impact Analysis: This is a catastrophic misconfiguration. A remote attacker requires no technical skill to exploit this; they simply connect a VNC client, type "password", and immediately gain full graphical control of the server's desktop interface.
Remediation: The immediate fix is to reconfigure the VNC service to require a complex, cryptographically secure passphrase. Furthermore, access to port 5900 should be restricted at the firewall level (iptables) to only allow connections from dedicated administrator IP addresses.

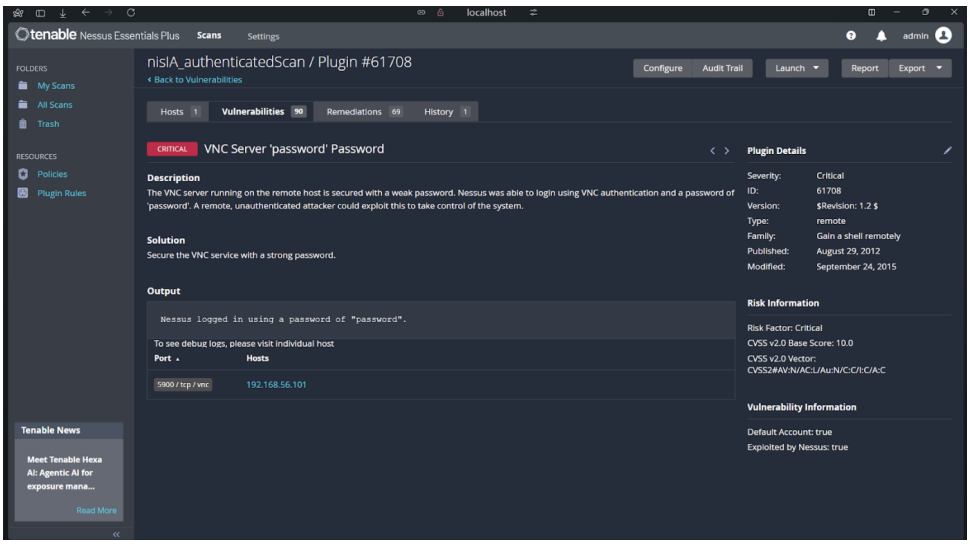

Figure 7: Detailed plugin output highlighting the VNC weak password and providing remediation advice.

7.2 Finding 2: Bind Shell Backdoor (CVSS 9.8)

Description: The scanner detected that a command shell was actively listening on TCP port 1524, and crucially, it did not prompt for any authentication or login credentials.
Impact Analysis: This is a classic "backdoor" mechanism. If an attacker connects to this port using a tool like Netcat (`nc 192.168.56.101 1524`), they are instantly dropped into a root-level terminal. They have absolute control over the operating system, can delete files, install malware, or pivot to attack other machines on the subnet.

Remediation: The system administrator must immediately identify the rogue Process ID (PID) binding port 1524 and kill it. Because a backdoor implies the system was previously compromised, a full forensic investigation is required, and the most secure remediation is wiping the machine and restoring from a known-good backup.

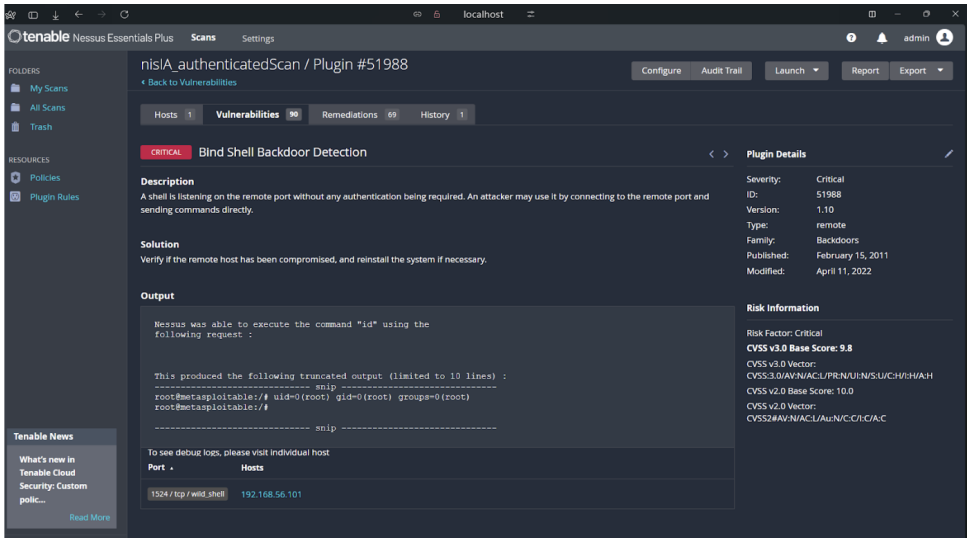

Figure 8: Detailed output showing Nessus successfully exploiting the Bind Shell backdoor to execute the 'id' command.

8 DevSecOps: Security Automation via Nessus REST API

Modern Network Security relies heavily on automation to integrate scanning into continuous integration/continuous deployment (CI/CD) pipelines. While the free version of Nessus Essentials locks the GUI feature for scheduling scans, we bypassed this limitation by interacting directly with the **Nessus REST API**.

From the user account settings, we generated cryptographic **Access** and **Secret API Keys**. We then authored a Python script utilizing the `requests` library. This script passes the API keys in the HTTP headers to authenticate, makes a GET request to the `/scans` endpoint, parses the JSON response, and outputs a formatted status report of all currently configured scans on the server.

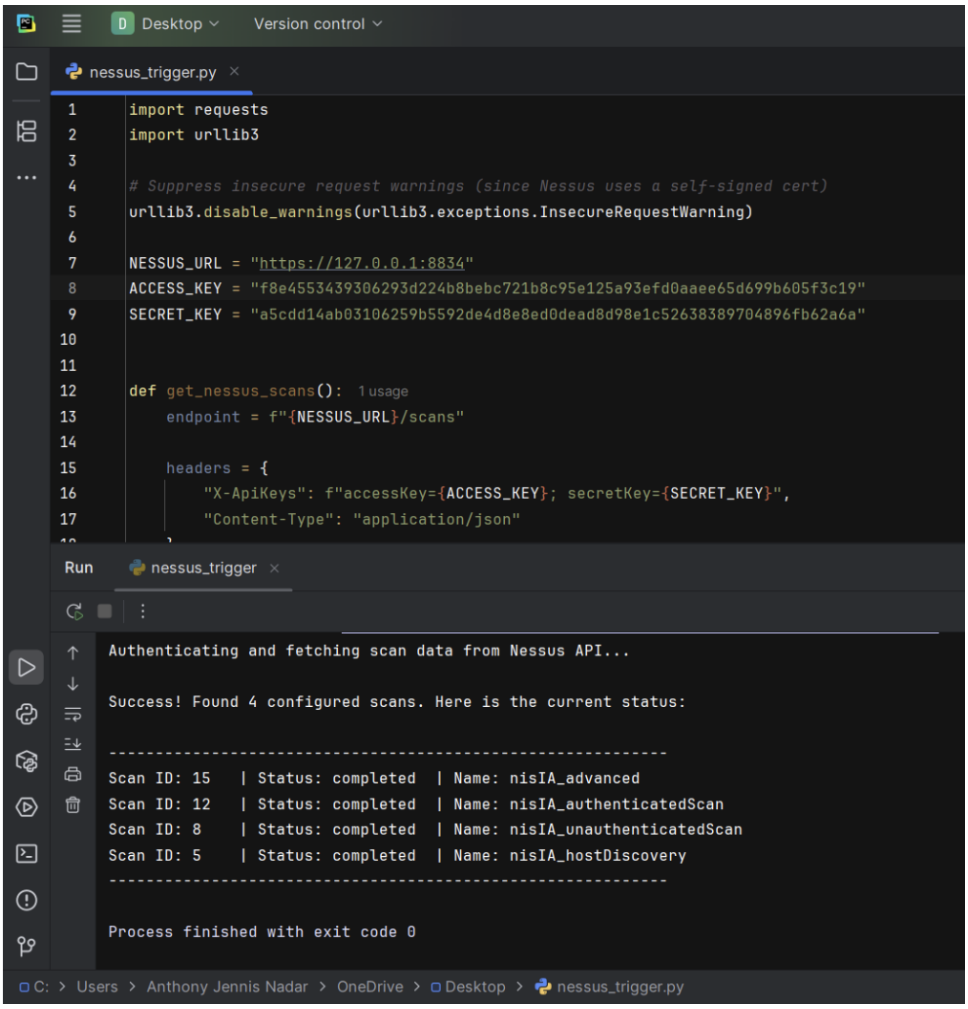

Figure 9: Successful execution of the Python automation script querying the Nessus API for real-time scan statuses.

```python
import requests
import urllib3

# Suppress insecure request warnings (Nessus uses a self-signed cert on localhost)
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

NESSUS_URL = "https://127.0.0.1:8834"
ACCESS_KEY = "f8e4553439306293d224b8bebc721b8c95e125a93efd0aaee65d699b605f3c19"
SECRET_KEY = "a5cdd14ab03106259b5592de4d8e8ed0dead8d98e1c52638389704896fb62a6a"

def get_nessus_scans():
    endpoint = f"{NESSUS_URL}/scans"
    headers = {
        "X-ApiKeys": f"accessKey={ACCESS_KEY}; secretKey={SECRET_KEY}",
        "Content-Type": "application/json"
    }
    print("Authenticating and fetching scan data from Nessus API...\n")
    try:
        # verify=False bypasses SSL certificate validation for localhost
        response = requests.get(endpoint, headers=headers, verify=False)
        if response.status_code == 200:
            scans = response.json().get("scans", [])
            if not scans:
                print("No scans found.")
            else:
                print(f"Success! Found {len(scans)} configured scans. Current status:\n")
                print("-" * 65)
                for scan in scans:
                    name = scan.get("name")
                    status = scan.get("status")
                    scan_id = scan.get("id")
                    print(f"Scan ID: {scan_id:<4} | Status: {status:<10} | Name: {name}")
                print("-" * 65)
        else:
            print(f"Failed to fetch scans. HTTP Status Code: {response.status_code}")
    except Exception as e:
        print(f"A network/API error occurred: {e}")

if __name__ == "__main__":
    get_nessus_scans()
```

Listing 1: Python Automation Script (`nessus_trigger.py`) used for the REST API.

9 Conclusion

This project successfully validated the end-to-end lifecycle of infrastructure vulnerability management. By configuring an isolated Metasploitable 2 environment, we securely demonstrated the capabilities of Tenable Nessus.

The core takeaway from the implementation is the vast disparity between unauthenticated and authenticated scanning; granting the scanner internal SSH credentials exponentially increased threat visibility, proving white-box testing is mandatory for internal compliance. Furthermore, by identifying high-risk vulnerabilities like open Bind Shells and weak VNC passwords, we practically applied risk remediation theory. Finally, the successful implementation of the Python API script proved that enterprise security tools can be seamlessly automated, laying the groundwork for modern DevSecOps practices.

10 References

References

[1] Tenable, Inc. (2024). *Tenable Nessus Essentials: Vulnerability Scanner*. Retrieved from `https://www.tenable.com/products/nessus/nessus-essentials`

[2] Oracle Corporation. (2024). *Oracle VM VirtualBox - Virtualization Platform*. Retrieved from `https://www.virtualbox.org/`

[3] Rapid7. (2012). *Metasploitable 2: Exploitation Training Environment*. Retrieved from `https://docs.rapid7.com/metasploit/metasploitable-2/`

[4] Python Software Foundation. (2024). *Requests: HTTP for Humans*. Retrieved from `https://requests.readthedocs.io/en/latest/`